

Python Concurrency Using Threads - Cheat Sheet

This cheat sheet provides a quick reference for using Python's threading module to implement concurrency. It includes examples for creating threads, synchronization techniques, and tips for avoiding common pitfalls.

1. Creating and Starting Threads

Threads allow multiple tasks to run seemingly in parallel within the same process. Use the **threading** module to create and start threads.

Example:

```
import threading

def worker():
    print('Worker thread running')

t = threading.Thread(target=worker)
t.start()
t.join()  # Wait for the thread to finish
```

2. Thread Identification and Naming

You can give threads names or identify them for debugging and logging.

Example:

```
import threading

def show_name():
    print(f'Running thread: {threading.current_thread().name}')

t = threading.Thread(target=show_name, name='MyWorker')
t.start()
```

3. Synchronizing Threads with Locks

When multiple threads modify shared data, use locks to avoid race conditions.

Example:

```
import threading
```

```

lock = threading.Lock()
counter = 0

def increment():
    global counter
    for _ in range(1000):
        with lock:
            counter += 1

threads = [threading.Thread(target=increment) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()

print('Counter:', counter)

```

4. Using ThreadPoolExecutor

The `concurrent.futures.ThreadPoolExecutor` simplifies running functions across multiple threads.

Example:

```

from concurrent.futures import ThreadPoolExecutor

def square(n):
    return n * n

with ThreadPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(square, range(10)))
print(results)

```

5. Daemon Threads

Daemon threads run in the background and terminate when the main program exits.

Example:

```

import threading

def background_task():
    while True:
        print('Running in background...')

t = threading.Thread(target=background_task, daemon=True)
t.start()

```

6. Common Threading Utilities

- `threading.active_count()` — Returns the number of running threads.
- `threading.enumerate()` — Lists all active Thread objects.
- `threading.current_thread()` — Returns the current Thread instance.

7. Thread Communication with Queue

Use `queue.Queue` to safely share data between threads without explicit locks.

Example:

```
import threading, queue

q = queue.Queue()

def producer():
    for i in range(5):
        q.put(i)

def consumer():
    while not q.empty():
        print('Consumed:', q.get())

threading.Thread(target=producer).start()
threading.Thread(target=consumer).start()
```

8. Understanding the GIL

The Global Interpreter Lock (GIL) prevents multiple native threads from executing Python bytecode simultaneously. Threading is best for I/O-bound tasks but not CPU-bound ones.

Use the ``multiprocessing`` module for true parallelism in CPU-heavy tasks.

9. Best Practices

- Use **with lock:** for safe and concise synchronization.
- Keep threads short-lived and task-focused.
- Avoid shared mutable data when possible.
- Prefer **ThreadPoolExecutor** for managing thread pools.
- Use `queue.Queue` instead of manual synchronization when sharing data.